

SecureSeat / CricNexus Interview Preparation Guide

Generated from the repository analysis for interview readiness

1. Project Summary

SecureSeat is a full-stack cricket ticketing platform focused on safe booking under high concurrency, dynamic pricing, and biometric anti-scalping. The product is built as a tri-service architecture: a React frontend, a Node/Express backend, and a Python FastAPI face-recognition microservice. The project emphasizes preventing seat duplication using Redis distributed locks and binding the ticket to a face embedding so a resold ticket is harder to reuse.

The business goal is to reduce ticket scalping, support fast booking during high traffic, and showcase a realistic booking pipeline with fake payment simulation for internal testing.

2. Core Architecture

The system is composed of three logical layers:

- Frontend: React + Vite + React Router + WebRTC/Webcam for capture and 3D/2D stadium visualization.
- Backend: Node.js + Express + PostgreSQL + Redis for lock coordination and transaction integrity.
- AI Service: Python + FastAPI + OpenCV YuNet/SFace to extract embeddings from uploaded images.

Architecture flow

1. User selects match/seat in the frontend. 2. Backend acquires a short-lived Redis lock on the seat. 3. User captures a face using webcam. 4. Payment is simulated and verified in the backend. 5. Ticket is written to PostgreSQL with face_embedding stored as a vector. 6. Security staff can retrieve the ticket and verify live face similarity using the AI service.

3. Major Components and Why They Were Chosen

Frontend

React was used for reusable components, simple state management, and navigation across login, checkout, stadium map, security scan, and user portal. Vite was chosen for fast local development and fast HMR. Three.js was integrated for visual stadium rendering and a more realistic seat browsing experience. react-webcam was used for direct browser camera capture without requiring a native app.

Backend

Node.js with Express is used because the app is API-driven and the team needs JavaScript across the stack. PostgreSQL was selected because booking and payment data need ACID guarantees. Redis is used as a temporary lock store to prevent race conditions during checkout. JWT-based auth plus cookies enables secure session handling and easy integration with the React app.

AI Service

A separate Python microservice isolates expensive image processing from the main API. YuNet is a fast face detector and SFace is a compact recognition model, which is suitable for a lightweight demo and local development. The service returns a 128-dimensional embedding instead of storing raw images, reducing privacy risk and storage cost.

4. Key Implementation Snippets

A. Redis seat lock with atomic NX EX

```
const lockKey = `seat_lock:${matchId}:${seatId}`;
const lockAcquired = await redisClient.set(lockKey, String(userId), {
  NX: true,
  EX: 600
});

if (!lockAcquired) {
  const currentOwner = await redisClient.get(lockKey);
  if (currentOwner === String(userId)) {
    const ttl = await redisClient.ttl(lockKey);
    return res.status(200).json({ message: 'Seat lock resumed.' });
  }
  return res.status(409).json({ message: 'Seat is currently locked by another user.' });
}
```

Why it matters: NX makes the lock creation atomic, and EX automatically releases the lock after 600 seconds. This is the main anti-double-booking mechanism during peak traffic.

B. confirmBooking flow: validate payment then store embedding

```
const paymentCheck = await dbPool.query(
  'SELECT status, amount FROM Payments WHERE id = $1 AND user_id = $2',
  [paymentId, userId]
);

if (paymentCheck.rows[0].status !== 'completed') {
  return res.status(400).json({ message: 'Payment not completed.' });
}

const aiResponse = await fetch(`${process.env.AI_SERVICE_URL}/generate-embedding`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ image_base64: photoBase64 })
});

const vectorString = `[${faceEmbedding.join(',')}]`;
await client.query(
  `INSERT INTO Tickets (user_id, match_id, seat_id, payment_id, status, face_embedding, ticket_price)
  VALUES ($1, $2, $3, $4, 'Booked', $5::vector, $6) RETURNING *`,
  [userId, matchId, seatId, paymentId, vectorString, ticketPrice]
);
```

This shows the booking commit path: payment validity, biometric encoding, and database persistence in a transaction. If any step fails, the whole booking flow should not leave partial state behind.

C. AI similarity check at security scan

```
@router.post('/verify-face')
async def verify_face(payload: VerifyPayload):
    live_img_array = decode_base64_image(payload.live_image_base64)
    live_vector = generate_face_embedding(live_img_array)
    similarity = calculate_cosine_similarity(live_vector, payload.stored_embedding)
    is_match = similarity >= 0.70
    return {
        'match': is_match,
        'similarity_score': round(similarity, 4)
    }
```

A threshold of 0.70 is a simple demonstration rule. In production, it would be tuned with calibration datasets and could be made adaptive per camera quality.

D. Dynamic pricing calculation

```
function computeDynamicPrice(basePrice, {
  daysLeft = 30, occupancyRatio = 0, standType = 'GENERAL',
```

```

    totalSeats = 1000, availableSeats = 1000, recentBookings24h = 0,
  } = {}) {
    let timeFactor = daysLeft > 30 ? 0.9 : daysLeft > 14 ? 1.0 : daysLeft > 7 ? 1.1 :
      daysLeft > 3 ? 1.25 : daysLeft > 1 ? 1.45 : 1.6;

    let scarcityFactor = occupancyRatio < 0.3 ? 1.0 : occupancyRatio < 0.5 ? 1.05 :
      occupancyRatio < 0.7 ? 1.15 : occupancyRatio < 0.85 ? 1.3 :
      occupancyRatio < 0.95 ? 1.5 : 1.75;

    const tierMap = { VIP: 1.15, PREMIUM: 1.08, COVERED: 1.03, GENERAL: 1.0, OPEN: 0.95 };
    const tierFactor = tierMap[standType?.toUpperCase()] ?? 1.0;

    const raw = timeFactor * scarcityFactor * tierFactor;
    const multiplier = Math.round(Math.min(2.5, Math.max(0.85, raw)) * 100) / 100;
    return { price: Math.round(basePrice * multiplier), basePrice, multiplier };
  }

```

This pricing logic is useful for explaining yield management. It balances demand spikes with fairness, but it is heuristic and could be replaced by a more advanced model using historical sales data.

5. Database Schema Highlights

The schema is normalized around stadium structure and event configuration. Key tables include Users, Stadiums, Stands, Blocks, Seats, Matches, Match_Stands_Config, Payments, Tickets, and Bank. The Tickets table stores a vector(128) embedding. The unique index on (match_id, seat_id) prevents multiple active bookings for the same physical seat during a match.

```

CREATE TABLE Tickets (
  id SERIAL PRIMARY KEY,
  user_id INT REFERENCES Users(id),
  match_id INT REFERENCES Matches(id),
  seat_id INT REFERENCES Seats(id),
  payment_id INT REFERENCES Payments(id),
  status VARCHAR(20),
  face_embedding vector(128),
  ticket_price DECIMAL(10, 2),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE UNIQUE INDEX unique_active_ticket_per_seat
ON Tickets(match_id, seat_id)
WHERE status IN ('Locked', 'Booked', 'verified');

```

Why vector column matters: It allows biometric matching in SQL queries or through application code; however, storing high-dimensional embeddings in a general-purpose relational DB is fine for a demo, but enterprise scale may require specialized vector search indexes or a separate vector database.

6. Important Workflow Explanation

Booking lifecycle

1. Lock a seat using Redis. 2. Confirm payment using the fake gateway. 3. Call AI service to compute face embedding. 4. Store ticket with payment reference in PostgreSQL and release lock. 5. Security lookup fetches the stored embedding and verifies live face in real time.

This flow shows a good understanding of high-concurrency systems, user auth, and image-based identity. It also demonstrates how to separate concerns to keep the core booking path reliable.

7. Trade-offs and Design Decisions

- Redis lock vs database lock: Redis is faster and scales better for temporary seat holds, but lock expiry adds a timeout risk if users take too long to checkout.
- Fake payment gateway: This is excellent for demos and QA, but it is not a production-grade payment processor. It avoids real money risk but loses real bank-level reconciliation.
- Vector embeddings in PostgreSQL: Easy to integrate with existing infrastructure, but not as optimized for large-scale similarity search as dedicated vector stores.

- Separate AI microservice: Decouples heavy image processing from the main app, but introduces network latency and a new deployment dependency.
- JWT in secure cookies: Good for session management and CSRF mitigation when configured well, but requires careful cookie policy and environment setup.

8. Likely Interview Questions and Strong Answers

Q1. How did you prevent double booking?

I used a Redis lock with SET key value NX EX 600. This means the first request to book a seat acquires the lock atomically. If another user tries to reserve the same seat during the 10-minute window, Redis rejects the request and the backend returns a 409. The lock is automatically removed after expiry, so seats are released even if the user leaves the checkout page.

Q2. Why is the AI service separate from the backend?

The face embedding generation is CPU-intensive and model-dependent. By isolating it in a FastAPI service, the booking backend stays responsive for authentication, payment, and database operations. If the AI service is down, the code falls back to a placeholder embedding for testing, which is practical for demos but not production-safe.

Q3. Why store a face embedding instead of the original image?

A vector embedding is a compact, privacy-friendly representation of a face. It reduces storage and simplifies similarity comparisons. In the security check, the system computes a cosine similarity between the live image embedding and the stored embedding. This makes the biometric identity check more scalable than storing large raw images.

Q4. What is the biggest trade-off in this project?

The largest trade-off is balancing realistic functionality with demo simplicity. The fake payment gateway and fallback embeddings help demonstrate the product flow, but they are not suitable for real production without bank-grade payment providers, better fraud monitoring, and a hardened AI pipeline with proper threshold tuning and liveness detection.

Q5. How would you improve this system for production?

I would replace the fake gateway with a real payment provider, add idempotency keys for bookings and payments, use a dedicated vector database or pgvector indexing for large-scale face searches, add liveness detection, strengthen role-based security, use rate limits, and introduce monitoring with logs and alerts around lock contention and payment failures.

9. Suggested Talking Points During the Interview

- The project is built with separation of concerns in mind.
- The lock is a practical example of CAP-style trade-offs in a booking system.
- The face embedding flow gives a clear story around anti-scalping.
- The project is flexible because the UI supports multiple views: photo and 3D stadium.
- The schema is designed to give strong integrity for a ticketing domain.

10. Conclusion

This project is strong for an interview because it touches concurrency control, secure payments, a microservice architecture, vector-based biometric verification, and a polished frontend. The most important thing to emphasize is the reasoning behind each design choice and the trade-offs you understand, not just the final implementation.